Microservices Implementation Guide

Complete Step-by-Step Enterprise Architecture Reference

Real-Time Order Processing System — Build from Scratch

Version: 2.0 | **Date:** March 2026 **Classification:** Internal — Architecture Reference **Spring Boot:** 3.x | **Java:** 17+

Table of Contents

1. Executive Summary
2. Architecture Overview
3. Development Order Strategy
4. Infrastructure Setup
5. Service-by-Service Implementation Guide
6. Event-Driven Architecture (Kafka)
7. Caching Strategy (Redis)
8. Security Architecture (JWT)
9. Design Patterns Used
10. Docker & Deployment
11. Production Considerations
12. Common Pitfalls & Solutions
13. Interview Questions
14. Future Enhancements

---

1. Executive Summary

1.1 What Are We Building?

A **Real-Time Order Processing System** — an enterprise-grade, event-driven microservices application that handles the complete lifecycle: user registration, product catalog management, order placement, inventory reservation, payment processing, and notification delivery — all running as independent, loosely coupled services.

1.2 System Components

```
+-------------------+-----------+-------------------------------------------+
| Component         | Tech      | Purpose                                   |
+-------------------+-----------+-------------------------------------------+
| Eureka Server     | Spring    | Service discovery — services register and |
|                   | Cloud     | find each other dynamically               |
+-------------------+-----------+-------------------------------------------+
| API Gateway       | Spring    | Single entry point — JWT validation, routing,|
|                   | Cloud GW  | rate limiting, circuit breakers           |
+-------------------+-----------+-------------------------------------------+
| Auth Service      | Spring    | User registration, login, JWT token generation|
|                   | Security  | BCrypt password hashing                   |
+-------------------+-----------+-------------------------------------------+
| User Service      | Spring    | User profile management (CRUD)            |
|                   | Boot      | Publishes user-events to Kafka            |
+-------------------+-----------+-------------------------------------------+
```

```
| Product Service  | Spring    | Product catalog CRUD, Redis caching          |
|                  | Boot      | Publishes product-created events to Kafka    |
+------------------+-----------+----------------------------------------------+
| Order Service    | Spring    | Order placement, status tracking             |
|                  | Boot      | Publishes order-created/cancelled to Kafka   |
+------------------+-----------+----------------------------------------------+
| Inventory Service| Spring    | Stock management, Redis distributed locks    |
|                  | Boot      | Consumes orders, produces inventory events   |
+------------------+-----------+----------------------------------------------+
| Payment Service  | Spring    | Payment processing (simulated)               |
|                  | Boot      | Consumes inventory events, produces payment  |
+------------------+-----------+----------------------------------------------+
| Notification Svc | Spring    | Notification + audit log management          |
|                  | Boot      | Consumes payment + audit events from Kafka   |
+------------------+-----------+----------------------------------------------+
| Frontend         | React     | User-facing web application                  |
+------------------+-----------+----------------------------------------------+
| Common Events    | Java      | Shared event POJOs (OrderEvent, PaymentEvent)|
|                  | Module    | Ensures type consistency across services      |
+------------------+-----------+----------------------------------------------+
```

1.3 Technology Stack

```
Backend:   Java 17+, Spring Boot 3.x, Spring Cloud, Spring Security
Messaging: Apache Kafka (event-driven communication)
Caching:   Redis 7 (caching, distributed locks, rate limiting)
Database:  MySQL 8.0 (database-per-service pattern)
Gateway:   Spring Cloud Gateway (reactive, non-blocking)
Discovery: Eureka Server (service registry)
Resilience: Resilience4j (circuit breaker, rate limiter, retry)
Build:     Maven (multi-module parent POM)
Container: Docker + Docker Compose
Frontend:  React (Nginx in production)
```

---

2. Architecture Overview

2.1 High-Level Architecture Diagram

```
+-------------------------------------------------------------------------+
|                                                                          |
|    ┌──────────────────────────────────────────────────────────────┐      |
|    │                      FRONTEND (React :3000)                  │      |
|    │   Login / Register / Products / Orders / Cart / Dashboard    │      |
|    └──────────────────────────┬───────────────────────────────────┘      |
|                               │ HTTP (REST)                              |
|                               ▼                                          |
|    ┌──────────────────────────────────────────────────────────────┐      |
|    │                  API GATEWAY (Spring Cloud Gateway :8080)    │      |
|    │                                                              │      |
|    │  ┌──────────┐ ┌─────────┐ ┌──────────┐ ┌──────────────┐     │      |
```

```
│    │  │ JWT Filter │ │ Circuit  │ │ Rate Limiter │ │ Load Balancer   │  │      │
│    │  │ Validate + │ │ Breaker  │ │ (Resilience4j│ │ (lb:// + Eureka)│  │      │
│    │  │ Add Headers│ │ Fallback │ │ + Redis)     │ │                 │  │      │
│    │  └────────────┘ └──────────┘ └──────────────┘ └─────────────────┘  │      │
│    └──────────────────────────────┬──────────────────────────────────────┘      │
│                   ┌───────────────┼────────────────┐                            │
│                   ▼               ▼                ▼                            │
│    ┌────────────────┐ ┌──────────────┐ ┌────────────────┐                      │
│    │  AUTH SERVICE  │ │ USER SERVICE │ │ PRODUCT SERVICE │                      │
│    │  (:8081)       │ │ (:8082)      │ │ (:8083)         │                      │
│    │                │ │              │ │                 │                      │
│    │  /api/auth/*   │ │ /api/users/* │ │ /api/products/* │                      │
│    │  signup, login │ │ profile CRUD │ │ catalog CRUD    │                      │
│    │  JWT generation│ │              │ │ @Cacheable(Redis)│                     │
│    └────────┬───────┘ └──────┬───────┘ └────────┬───────┘                      │
│             │                │                  │                              │
│             ▼                ▼                  ▼                              │
│        [auth_db]         [user_db]          [product_db]                        │
│                                                  │                              │
│                                          Kafka: product-created                 │
│                                                  │                              │
│    ┌────────────────┐ ┌──────────────────┐      │                              │
│    │ ORDER SERVICE  │ │ INVENTORY SERVICE │◄─────┘                              │
│    │ (:8084)        │ │ (:8085)          │                                     │
│    │                │ │                  │                                     │
│    │ /api/orders/*  │ │ /api/inventory/* │                                     │
│    │ place order    │ │ stock management │                                     │
│    │                │ │ Redis dist. lock │                                     │
│    └────────┬───────┘ └────────┬─────────┘                                     │
│             │                  │                                               │
│        Kafka: order-created    │ Kafka: inventory-updated                      │
│             │                  │          │                                    │
│             └──────────► INVENTORY ◄───────┘                                    │
│                                                                                 │
│    ┌────────────────┐ ┌──────────────────┐                                     │
│    │ PAYMENT SVC    │ │ NOTIFICATION SVC │                                     │
│    │ (:8086)        │ │ (:8087)          │                                     │
│    │                │ │                  │                                     │
│    │ Consumes inv.  │ │ Consumes payment │                                     │
│    │ events, pays   │ │ events + audit   │                                     │
│    └────────┬───────┘ └──────────────────┘                                     │
│             │                                                                   │
│  ┌──────────┼──────────── INFRASTRUCTURE ──────────────────┐                   │
│  │          │                                               │                   │
│  │  ┌───────┴──────┐ ┌──────────┐ ┌──────────────────┐    │                   │
│  │  │    KAFKA     │ │  REDIS   │ │ EUREKA SERVER    │    │                   │
│  │  │ :9092        │ │  :6379   │ │ :8761            │    │                   │
│  │  │ 8 topics     │ │ Cache +  │ │ Service registry │    │                   │
│  │  │ ZooKeeper    │ │ Locks +  │ │ + discovery      │    │                   │
│  │  │ :2181        │ │ RateLimit│ │                  │    │                   │
│  │  └──────────────┘ └──────────┘ └──────────────────┘    │                   │
│  └─────────────────────────────────────────────────────────┘                   │
```

```
│                                                                          │
│        ┌──────────────────────────────────────────────┐                  │
│        │              MySQL 8.0 (:3306)               │                  │
│        │  auth_db | user_db | product_db | order_db   │                  │
│        │  inventory_db | payment_db | notification_db │                  │
│        └──────────────────────────────────────────────┘                  │
+──────────────────────────────────────────────────────────────────────────+
```

2.2 Request Flow: Frontend → Gateway → Service

```
USER ACTION: "Buy Laptop" (clicks Place Order button)


  ┌──────────┐     ┌──────────────┐     ┌──────────────┐
  │ Frontend │────▶│  API Gateway │────▶│ Order Service│
  │ React    │     │  :8080       │     │ :8084        │
  └──────────┘     └──────────────┘     └──────────────┘
       │                 │                     │
  1. POST /api/orders   2. JWT Filter:        3. Save order to
     {productId: 1,        - Extract Bearer      order_db
      quantity: 2}          - Validate JWT     4. Publish to Kafka
     Authorization:        - Add X-User-Name      topic: order-created
     Bearer eyJhbG...      - Add X-User-Role      key: productId
                         3. Route: lb://
                            order-service
                         4. Circuit Breaker
                            (fallback if down)
```

2.3 Event-Driven Flow (Complete Order Lifecycle)

```
STEP 1: Order Placed
  Order Service → Kafka [order-created] → Inventory Service

STEP 2: Inventory Reserved
  Inventory Service → Redis Lock (SETNX) → Check stock → Reserve
  Inventory Service → Kafka [inventory-updated] → Order Service (status: RESERVED)
                                                 → Payment Service

STEP 3: Payment Processed
  Payment Service → Process payment → Save to payment_db
  Payment Service → Kafka [payment-success] → Order Service (status: CONFIRMED)
                                            → Notification Service

STEP 4: Notification Sent
  Notification Service → Save notification to notification_db

COMPENSATION (if stock insufficient):
  Inventory Service → Kafka [payment-failure] → Order Service (status: CANCELLED)
                                               → Notification Service (alert user)
```

2.4 Authentication Flow

```
SIGNUP FLOW:
  Frontend → POST /api/auth/signup {username, email, password, role}
  Gateway → routes to Auth Service (no JWT required for /api/auth/*)
  Auth Service → BCrypt hash password → save to auth_db → return success

LOGIN FLOW:
  Frontend → POST /api/auth/login {username, password}
  Auth Service → find user → BCrypt.matches(password, hash) → generate JWT
  Frontend ← {token: "eyJhbG...", username, role, message}

AUTHENTICATED REQUEST:
  Frontend → GET /api/products (Authorization: Bearer eyJhbG...)
  Gateway → JwtAuthenticationFilter:
    1. Extract token from "Bearer " prefix
    2. jwtUtil.isTokenValid(token) → verify signature + expiry
    3. Extract username + role from claims
    4. Add headers: X-User-Name, X-User-Role
    5. Forward to Product Service
  Product Service → reads X-User-Name header → knows who is calling
```

2.5 Synchronous vs Asynchronous Communication

```
SYNCHRONOUS (REST via Gateway):
  Frontend ──REST──▶ Gateway ──REST──▶ Service ──REST──▶ Response
  Used for: User-facing operations (login, get products, place order)
  WHY: User expects immediate response.

ASYNCHRONOUS (Kafka Events):
  Order Service ──event──▶ Kafka ──event──▶ Inventory Service
  Used for: Inter-service workflows (order → inventory → payment → notification)
  WHY: Loose coupling, fault tolerance, scalability.

WHY NOT ALL SYNCHRONOUS?
  Problem: Order Service calls Inventory Service (REST)
           → Inventory Service calls Payment Service (REST)
           → Payment Service calls Notification Service (REST)
           → If ANY service is down: entire chain fails → user gets 500 error
           → Tight coupling: Order Service MUST know Inventory Service URL

  Solution: Event-driven
           → Order Service publishes event → done (returns 200 to user immediately)
           → Inventory Service consumes when ready (can be down temporarily)
           → Kafka retains messages (no data loss)
           → Services don't know about each other (loose coupling)
```

2.6 Why This Architecture is Scalable

```
1. HORIZONTAL SCALING:
   Each service can be scaled independently.
   Order Service overloaded? → docker compose up --scale order-service=5
```

```
   → Eureka discovers new instances → Gateway load-balances across all 5.

2. DATABASE ISOLATION:
   Each service owns its database.
   Product Service DB slow? Scale it without affecting Order Service.

3. ASYNC PROCESSING:
   Kafka partitions enable parallel consumption.
   6 partitions × 6 consumers = 6x throughput.

4. CACHING:
   Redis absorbs 90% of read traffic.
   Product Service: 20x fewer DB queries.

5. CIRCUIT BREAKERS:
   One service failure doesn't cascade.
   Product Service down? Gateway returns fallback response.

6. STATELESS SERVICES:
   JWT tokens are self-contained.
   Any instance can handle any request (no session affinity).
```

---

3. Development Order Strategy

3.1 The Correct Build Sequence

```
WHY ORDER MATTERS:
  You cannot build Order Service before Kafka is running.
  You cannot build Product Service before MySQL is ready.
  You cannot build API Gateway before Eureka knows about services.
  You cannot test JWT validation before Auth Service generates tokens.

THE DEPENDENCY TREE:
  Level 0: Infrastructure (MySQL, Redis, Kafka, ZooKeeper)
  Level 1: Common Events Module (shared DTOs)
  Level 2: Eureka Server (service registry)
  Level 3: Auth Service (JWT generation — other services validate tokens)
  Level 4: API Gateway (routes requests, validates JWT)
  Level 5: Domain Services (User, Product, Order, Inventory, Payment, Notification)
  Level 6: Frontend (consumes API endpoints)
```

Step 1 — Infrastructure Setup (Docker)

```
WHAT:  MySQL 8.0, Redis 7, Apache Kafka + ZooKeeper
WHY:   Every service depends on at least one infrastructure component
HOW:   docker compose up -d mysql redis zookeeper kafka
VERIFY: mysqladmin ping, redis-cli PING, kafka-topics --list

COMMON MISTAKE:
```

```
  Starting services before infrastructure is healthy.
  FIX: Use depends_on with condition: service_healthy
```

Step 2 — Common Events Module

```
WHAT:  Java library with shared event POJOs (OrderEvent, PaymentEvent, etc.)
WHY:   All services need consistent event schemas for Kafka communication
HOW:   Maven module with no Spring — pure Java classes + Lombok
BUILD: mvn install (installs to local .m2 repository)

COMMON MISTAKE:
  Forgetting to mvn install common-events before building other services.
  FIX: Always build common-events first — it's a dependency for 5 services.

CLASSES:
  OrderEvent, PaymentEvent, InventoryEvent, NotificationEvent,
  ProductCreatedEvent, AuditLogEvent, UserEvent, ProductEvent
```

Step 3 — Eureka Server

```
WHAT:  Service discovery server — all services register here
WHY:   API Gateway uses Eureka to find service instances dynamically
       Instead of hardcoding localhost:8084, Gateway uses lb://order-service
HOW:   Spring Boot app with @EnableEurekaServer + application.yml
PORT:  8761

COMMON MISTAKE:
  Starting application services before Eureka is healthy.
  FIX: Use healthcheck + depends_on condition: service_healthy
```

Step 4 — Auth Service

```
WHAT:  Handles user registration, login, and JWT token generation
WHY:   Built BEFORE Gateway and other services because:
       1. Gateway needs to validate JWT → must understand token format
       2. All services check X-User-Role header → Auth Service defines roles
       3. You need login to test ANY protected endpoint
PORT:  8081
DB:    auth_db

CLASSES (11):
  AuthServiceApplication, AuthController, AuthService,
  UserCredential (entity), UserCredentialRepository,
  JwtUtil, SecurityConfig, AuthRequest, AuthResponse,
  SignupRequest, GlobalExceptionHandler
```

Step 5 — API Gateway

```
WHAT:  Single entry point — JWT validation, routing, rate limiting, circuit breaker
WHY:   Built AFTER Auth Service because:
```

```
       1. JwtAuthenticationFilter validates tokens generated by Auth Service
       2. Routes to downstream services via Eureka (lb://)
       3. All subsequent testing goes through the Gateway
PORT:  8080

CLASSES (6):
  ApiGatewayApplication, JwtAuthenticationFilter (most critical),
  JwtUtil, CorsConfig, ResilienceConfig, FallbackController
```

Step 6 — User Service

```
WHAT:  User profile management (CRUD)
WHY:   After Auth because auth handles login/register; User handles profile
PORT:  8082
DB:    user_db
KAFKA: Produces user-events, audit-logs

CLASSES (10):
  UserServiceApplication, UserController, AdminUserController,
  UserService, UserProfile (entity), UserProfileRepository,
  UserProfileDto, UserEventProducer, SecurityConfig,
  GlobalExceptionHandler
```

Step 7 — Product Service

```
WHAT:  Product catalog CRUD with Redis caching
WHY:   Products must exist before orders can be placed
PORT:  8083
DB:    product_db
REDIS: @Cacheable, @CachePut, @CacheEvict (Cache-Aside pattern)
KAFKA: Produces product-created, product-events, audit-logs

CLASSES (11):
  ProductServiceApplication, ProductController, AdminProductController,
  ProductService, Product (entity), ProductRepository,
  ProductDto, RedisConfig, ProductEventProducer,
  SecurityConfig, GlobalExceptionHandler
```

Step 8 — Order Service

```
WHAT:  Order placement and status tracking
WHY:   After Product Service (orders reference products)
PORT:  8084
DB:    order_db
KAFKA: PRODUCER: order-created, order-cancelled
       CONSUMER: payment-success, payment-failure, inventory-updated

CLASSES (13):
  OrderServiceApplication, OrderController, AdminOrderController,
  OrderService, Order (entity), OrderRepository,
```

```
  OrderRequest, OrderResponse, OrderProducer,
  OrderStatusConsumer, InventoryStatusConsumer,
  SecurityConfig, GlobalExceptionHandler
```

Step 9 — Inventory Service

```
WHAT:  Stock management with distributed locking
WHY:   After Order Service (inventory consumes order-created events)
PORT:  8085
DB:    inventory_db
REDIS: StringRedisTemplate — SETNX distributed lock (30s TTL)
KAFKA: CONSUMER: order-created, order-cancelled, product-created
       PRODUCER: inventory-updated

CLASSES (13):
  InventoryServiceApplication, InventoryController, AdminInventoryController,
  InventoryService, Inventory (entity), InventoryRepository,
  ProcessedEvent (entity), ProcessedEventRepository,
  RedisLockConfig, InventoryConsumer, InventoryProducer,
  SecurityConfig, GlobalExceptionHandler
```

Step 10 — Payment Service

```
WHAT:  Payment processing (simulated)
WHY:   After Inventory (payment consumes inventory-updated events)
PORT:  8086
DB:    payment_db
KAFKA: CONSUMER: inventory-updated
       PRODUCER: payment-success, payment-failure

CLASSES (8):
  PaymentServiceApplication, PaymentController,
  PaymentService, Payment (entity), PaymentRepository,
  PaymentConsumer, PaymentProducer, GlobalExceptionHandler
```

Step 11 — Notification Service

```
WHAT:  Notification and audit log management
WHY:   Last domain service (consumes payment events — end of the saga)
PORT:  8087
DB:    notification_db
KAFKA: CONSUMER: payment-success, payment-failure, audit-logs

CLASSES (10):
  NotificationServiceApplication, NotificationController,
  AdminAuditController, NotificationConsumer, AuditLogConsumer,
  Notification (entity), AuditLog (entity),
  NotificationRepository, AuditLogRepository,
  SecurityConfig
```

Step 12 — Frontend Integration

```
WHAT:  React application with Nginx
WHY:   Built last — consumes all API endpoints
PORT:  3000

FLOW: React → http://localhost:8088/api/* → Gateway → Services
```

---

4. Infrastructure Setup

4.1 Docker Compose (Infrastructure Only)

```
# Start infrastructure components first
docker compose up -d mysql redis zookeeper kafka

# Wait for MySQL health check (30-60 seconds)
docker compose ps   # Wait until mysql shows "healthy"
```

4.2 MySQL Databases

The `init.sql` script creates 7 isolated databases on first startup:

```
CREATE DATABASE IF NOT EXISTS auth_db;
CREATE DATABASE IF NOT EXISTS user_db;
CREATE DATABASE IF NOT EXISTS product_db;
CREATE DATABASE IF NOT EXISTS order_db;
CREATE DATABASE IF NOT EXISTS inventory_db;
CREATE DATABASE IF NOT EXISTS payment_db;
CREATE DATABASE IF NOT EXISTS notification_db;

GRANT ALL PRIVILEGES ON auth_db.* TO 'orderuser'@'%';
-- ... (same for all databases)
FLUSH PRIVILEGES;
```

4.3 Maven Multi-Module Setup

```
backend/
├── pom.xml                    # Parent POM (manages shared dependencies)
├── common-events/             # Shared event POJOs (NO Spring)
├── eureka-server/
├── api-gateway/
├── auth-service/
├── user-service/
├── product-service/
├── order-service/
├── inventory-service/
├── payment-service/
└── notification-service/
```

Build order:

```
cd backend/common-events && mvn clean install    # Step 1: Install shared events
cd backend && mvn clean package -DskipTests       # Step 2: Build all services
```

---

5. Service-by-Service Implementation Guide

5.1 Common Events Module

PURPOSE: Centralized event schemas (DTOs) shared by all Kafka-producing and consuming services. Ensures type consistency — producer and consumer always agree on message format.

PROJECT STRUCTURE:

```
common-events/
├── pom.xml                              # Maven dependency (Lombok only)
└── src/main/java/com/orderprocessing/
    ├── commonevents/event/
    │   ├── AuditLogEvent.java           # Cross-cutting audit trail
    │   ├── ProductEvent.java            # Product lifecycle events
    │   └── UserEvent.java               # User lifecycle events
    └── events/
        ├── OrderEvent.java              # Order created/cancelled
        ├── PaymentEvent.java            # Payment success/failure
        ├── InventoryEvent.java          # Stock reserved/released
        ├── NotificationEvent.java       # Notification payloads
        └── ProductCreatedEvent.java     # New product initialization
```

KEY DESIGN: Every event has `eventId` (UUID)

```java
@Data @NoArgsConstructor @AllArgsConstructor
public class OrderEvent {
    private String eventId;        // UUID — unique per event (idempotency key)
    private Long orderId;
    private Long productId;
    private int quantity;
    private double totalAmount;
    private String status;         // PENDING, CONFIRMED, CANCELLED
    private String username;
    private LocalDateTime timestamp;
}
```

WHY `eventId`? Kafka guarantees at-least-once delivery. If a consumer crashes after processing but before committing the offset, the message is replayed → duplicate. `eventId` allows consumers to check: "Did I already process this?" → skip duplicate.

5.2 Eureka Server

PURPOSE: Service discovery — all microservices register here. The API Gateway uses Eureka to dynamically resolve service URLs (no hardcoding).

PROJECT STRUCTURE:

```
eureka-server/
├── pom.xml
├── Dockerfile
└── src/main/java/com/orderprocessing/eurekaserver/
    └── EurekaServerApplication.java
```

SINGLE CLASS:

```java
@SpringBootApplication
@EnableEurekaServer          // Activates Eureka Server
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

application.yml:

```yaml
server:
  port: 8761
spring:
  application:
    name: eureka-server
eureka:
  client:
    register-with-eureka: false      # Server doesn't register with itself
    fetch-registry: false            # Server doesn't fetch from itself
```

Key dependency:

```publication_info
<dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

5.3 Auth Service (11 Classes)

PURPOSE: Handles user registration, authentication, and JWT token generation. This is the ONLY service that creates JWT tokens — all other services consume them.

PROJECT STRUCTURE:

```
auth-service/
├── pom.xml
├── Dockerfile
```

```
└── src/main/java/com/orderprocessing/auth/
    ├── AuthServiceApplication.java
    ├── controller/
    │   └── AuthController.java        # REST: /api/auth/*
    ├── dto/
    │   ├── AuthRequest.java           # Login request {username, password}
    │   ├── AuthResponse.java          # Response {token, username, role}
    │   └── SignupRequest.java         # Register request {username, email, password}
    ├── exception/
    │   └── GlobalExceptionHandler.java
    ├── model/
    │   └── UserCredential.java        # JPA entity → auth_db.users
    ├── repository/
    │   └── UserCredentialRepository.java
    ├── security/
    │   ├── JwtUtil.java               # JWT generation + validation
    │   └── SecurityConfig.java        # BCrypt + endpoint security
    └── service/
        └── AuthService.java           # Business logic
```

CLASS-BY-CLASS EXPLANATION:

Layer: Controller

```java
@RestController
@RequestMapping("/api/auth")
public class AuthController {
    // POST /api/auth/signup → Register new user
    @PostMapping("/signup")
    public ResponseEntity<AuthResponse> signup(@Valid @RequestBody SignupRequest request) {
        AuthResponse response = authService.signup(request);
        return ResponseEntity.status(HttpStatus.CREATED).body(response);
    }

    // POST /api/auth/login → Authenticate and get JWT
    @PostMapping("/login")
    public ResponseEntity<AuthResponse> login(@Valid @RequestBody AuthRequest request) {
        AuthResponse response = authService.login(request);
        return ResponseEntity.ok(response);
    }

    // GET /api/auth/validate → Validate JWT token
    @GetMapping("/validate")
    public ResponseEntity<Map<String, Object>> validateToken(@RequestParam("token") String
token) {
        boolean isValid = authService.validateToken(token);
        return ResponseEntity.ok(Map.of("valid", isValid));
    }
}
```

WHY this class exists: Single responsibility — handles HTTP request/response mapping. No business logic here — delegates everything to AuthService.

Layer: Service

```
@Service
public class AuthService {
    // SIGNUP FLOW:
    // 1. Check username uniqueness
    // 2. Check email uniqueness
    // 3. Hash password (BCrypt)
    // 4. Save UserCredential to auth_db
    // 5. Return success response

    // LOGIN FLOW:
    // 1. Find user by username
    // 2. BCrypt.matches(provided, stored) → verify password
    // 3. Generate JWT: jwtUtil.generateToken(username, role)
    // 4. Return token + user info
}
```

Layer: Entity

```
@Entity @Table(name = "users")
public class UserCredential {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(unique = true, nullable = false)
    private String username;
    @Column(unique = true, nullable = false)
    private String email;
    @Column(nullable = false)
    private String password;              // BCrypt hash
    @Enumerated(EnumType.STRING)
    private Role role;                    // USER or ADMIN

    public enum Role { USER, ADMIN }
}
```

Layer: Security

```
@Configuration @EnableWebSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) {
        return http
            .csrf(csrf -> csrf.disable())                   // Stateless API
            .sessionManagement(session ->
                session.sessionCreationPolicy(STATELESS))    // No sessions
            .authorizeHttpRequests(auth -> auth
```

```
                .requestMatchers("/api/auth/**").permitAll()  // Open endpoints
                .anyRequest().authenticated())
            .build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();    // 12 rounds by default
    }
}
```

WHY BCrypt? Adaptive hashing — as hardware gets faster, increase rounds. Each hash includes a random salt → same password produces different hashes → rainbow tables useless.

5.4 API Gateway (6 Classes)

PURPOSE: Single entry point for ALL client requests. Validates JWT tokens, routes to services via Eureka discovery, applies circuit breakers, and handles CORS.

PROJECT STRUCTURE:

```
api-gateway/
├── pom.xml
├── Dockerfile
└── src/main/java/com/orderprocessing/gateway/
    ├── ApiGatewayApplication.java
    ├── config/
    │   ├── CorsConfig.java                # CORS for React frontend
    │   └── ResilienceConfig.java          # Circuit breaker config
    ├── controller/
    │   └── FallbackController.java        # Fallback responses when services are down
    ├── filter/
    │   └── JwtAuthenticationFilter.java   # THE MOST CRITICAL CLASS
    └── util/
        └── JwtUtil.java                   # JWT validation (shared secret)
```

CRITICAL CLASS — JwtAuthenticationFilter:

```
@Component
public class JwtAuthenticationFilter extends AbstractGatewayFilterFactory<Config> {

    private static final List<String> OPEN_ENDPOINTS = List.of(
            "/api/auth/signup", "/api/auth/login", "/api/auth/validate", "/fallback");

    @Override
    public GatewayFilter apply(Config config) {
        return (exchange, chain) -> {
            String path = exchange.getRequest().getURI().getPath();

            // 1. Skip auth for open endpoints
            if (isOpenEndpoint(path)) return chain.filter(exchange);
```

```java
            // 2. Skip JWT for CORS preflight (OPTIONS)
            if ("OPTIONS".equalsIgnoreCase(request.getMethod().name()))
                return chain.filter(exchange);

            // 3. Extract Bearer token
            String authHeader = request.getHeaders().getFirst(AUTHORIZATION);
            if (authHeader == null || !authHeader.startsWith("Bearer "))
                return onError(exchange, "Missing Authorization", UNAUTHORIZED);

            String token = authHeader.substring(7);

            // 4. Validate JWT
            if (!jwtUtil.isTokenValid(token))
                return onError(exchange, "Token expired or invalid", UNAUTHORIZED);

            // 5. Extract claims, add as headers for downstream
            String username = jwtUtil.extractUsername(token);
            String role = jwtUtil.extractRole(token);
            ServerHttpRequest modifiedRequest = request.mutate()
                    .header("X-User-Name", username)
                    .header("X-User-Role", role)
                    .build();

            return chain.filter(exchange.mutate().request(modifiedRequest).build());
        };
    }
}
```

WHY this filter is the most critical class:

1. ALL authenticated requests pass through it
2. A bug here → either all requests fail OR authentication is bypassed
3. It bridges Auth Service's JWT and downstream services' user context
4. CORS handling prevents frontend JavaScript errors

application.yml — Route Configuration:

```yaml
spring:
  cloud:
    gateway:
      routes:
        - id: auth-service
          uri: lb://auth-service           # Eureka load-balanced
          predicates:
            - Path=/api/auth/**
          filters:
            - name: CircuitBreaker
              args:
                name: authServiceCircuitBreaker
                fallbackUri: forward:/fallback/auth

        - id: product-service
```

```yaml
        uri: lb://product-service
        predicates:
          - Path=/api/products/**, /admin/products/**
        filters:
          - JwtAuthenticationFilter       # JWT validation
          - name: CircuitBreaker
            args:
              name: productServiceCircuitBreaker
              fallbackUri: forward:/fallback/generic
          - name: Retry
            args:
              retries: 3
              methods: GET
```

5.5 User Service (10 Classes)

PURPOSE: User profile management (CRUD). Separate from Auth Service to follow Single Responsibility — Auth handles credentials, User handles profiles.

PROJECT STRUCTURE:

```
user-service/
├── pom.xml
└── src/main/java/com/orderprocessing/userservice/
    ├── UserServiceApplication.java
    ├── config/
    │   └── SecurityConfig.java            # @PreAuthorize + role-based access
    ├── controller/
    │   ├── UserController.java            # /api/users/* (user self-service)
    │   └── AdminUserController.java       # /admin/users/* (admin management)
    ├── dto/
    │   └── UserProfileDto.java
    ├── exception/
    │   └── GlobalExceptionHandler.java
    ├── kafka/
    │   └── UserEventProducer.java         # Publishes user-events + audit-logs
    ├── model/
    │   └── UserProfile.java               # JPA entity → user_db
    ├── repository/
    │   └── UserProfileRepository.java
    └── service/
        └── UserService.java
```

KEY DESIGN — Role-Based Access:

```java
// AdminUserController.java — ADMIN only
@RestController
@RequestMapping("/admin/users")
public class AdminUserController {
    @GetMapping
    @PreAuthorize("hasRole('ADMIN')")
```

```
    public List<UserProfileDto> getAllUsers() { ... }
}

// UserController.java — Any authenticated user
@RestController
@RequestMapping("/api/users")
public class UserController {
    @GetMapping("/profile")
    public UserProfileDto getMyProfile(@RequestHeader("X-User-Name") String username) {
        return userService.getProfileByUsername(username);
    }
}
```

WHY X-User-Name header? The Gateway's JWT filter extracts the username from the JWT and injects it as a header. Downstream services don't need to parse JWT — they just read the header. This is a clean separation of concerns.

5.6 Product Service (11 Classes)

PURPOSE: Product catalog CRUD with Redis caching. Publishes `product-created` events to Kafka so Inventory Service can initialize stock.

PROJECT STRUCTURE:

```
product-service/
├── pom.xml
└── src/main/java/com/orderprocessing/productservice/
    ├── ProductServiceApplication.java
    ├── config/
    │   ├── RedisConfig.java              # CacheManager + JSON serializer
    │   └── SecurityConfig.java
    ├── controller/
    │   ├── ProductController.java        # /api/products/* (public catalog)
    │   └── AdminProductController.java   # /admin/products/* (ADMIN CRUD)
    ├── dto/
    │   └── ProductDto.java
    ├── exception/
    │   └── GlobalExceptionHandler.java
    ├── kafka/
    │   └── ProductEventProducer.java     # Publishes product-created events
    ├── model/
    │   └── Product.java                  # JPA entity → product_db
    ├── repository/
    │   └── ProductRepository.java
    └── service/
        └── ProductService.java           # Caching + Kafka publishing
```

CACHING STRATEGY (Cache-Aside with Spring Annotations):

```
@Service
public class ProductService {

    // READ — Cache-Aside pattern
```

```java
    @Cacheable(value = "product", key = "#id")
    public ProductDto getProductById(Long id) {
        log.info("CACHE MISS — Fetching from DB");
        return mapEntityToDto(repository.findById(id).orElseThrow());
    }

    // CREATE — Evict list cache (new product makes it stale)
    @CacheEvict(value = "products", allEntries = true)
    public ProductDto createProduct(ProductDto dto) {
        Product saved = repository.save(product);
        // Fire-and-forget Kafka event → Inventory Service initializes stock
        CompletableFuture.runAsync(() ->
            productEventProducer.publishProductCreatedEvent(event));
        return mapEntityToDto(saved);
    }

    // UPDATE — Update cache + evict list
    @CachePut(value = "product", key = "#id")
    @CacheEvict(value = "products", allEntries = true)
    public ProductDto updateProduct(Long id, ProductDto dto) { ... }

    // DELETE — Evict both caches
    @CacheEvict(value = {"product", "products"}, allEntries = true)
    public void deleteProduct(Long id) { ... }   // Soft delete (active=false)
}
```

RedisConfig — JSON Serialization + Per-Cache TTL:

```java
@Configuration
public class RedisConfig {
    @Bean
    public CacheManager cacheManager(RedisConnectionFactory factory) {
        RedisCacheConfiguration defaultConfig = RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(10))          // Default: 10 min
                .disableCachingNullValues()                 // Prevent cache penetration
                .serializeKeysWith(StringRedisSerializer)
                .serializeValuesWith(GenericJackson2JsonRedisSerializer);

        RedisCacheConfiguration listConfig = defaultConfig.entryTtl(Duration.ofMinutes(5));

        return RedisCacheManager.builder(factory)
                .cacheDefaults(defaultConfig)              // "product" → 10 min
                .withCacheConfiguration("products", listConfig)  // "products" → 5 min
                .build();
    }
}
```

5.7 Order Service (13 Classes)

PURPOSE: Order placement and lifecycle tracking. Publishes order events, consumes payment and inventory status events to update order status.

PROJECT STRUCTURE:

```
order-service/
├── pom.xml
└── src/main/java/com/orderprocessing/orderservice/
    ├── OrderServiceApplication.java
    ├── config/
    │   └── SecurityConfig.java
    ├── controller/
    │   ├── OrderController.java           # /api/orders/* (user orders)
    │   └── AdminOrderController.java      # /admin/orders/* (admin view)
    ├── dto/
    │   ├── OrderRequest.java              # {productId, quantity}
    │   └── OrderResponse.java             # {orderId, status, totalAmount}
    ├── exception/
    │   └── GlobalExceptionHandler.java
    ├── kafka/
    │   ├── OrderProducer.java             # Publishes order-created/cancelled
    │   ├── OrderStatusConsumer.java        # Consumes payment-success/failure
    │   └── InventoryStatusConsumer.java    # Consumes inventory-updated
    ├── model/
    │   └── Order.java                     # JPA entity → order_db
    ├── repository/
    │   └── OrderRepository.java
    └── service/
        └── OrderService.java
```

ORDER LIFECYCLE (Saga):

```
User places order → Order Service:
  1. Save order (status = PENDING) to order_db
  2. Publish OrderEvent to Kafka topic "order-created"
  3. Return 201 Created to user (async processing begins)

Inventory Service consumes order-created:
  4. Reserve stock (Redis lock + DB update)
  5. Publish InventoryEvent to "inventory-updated"

InventoryStatusConsumer receives inventory-updated:
  6. Update order status → INVENTORY_RESERVED or CANCELLED

Payment Service consumes inventory-updated:
  7. Process payment → publish payment-success or payment-failure

OrderStatusConsumer receives payment result:
  8. Update order status → CONFIRMED or PAYMENT_FAILED
```

PRODUCER — OrderProducer:

```java
@Component
public class OrderProducer {
    private final KafkaTemplate<String, OrderEvent> kafkaTemplate;
```

```
    public void sendOrderEvent(OrderEvent event) {
        // Key = productId → ensures all events for same product go to same partition
        // → guarantees ordering for that product's lifecycle
        kafkaTemplate.send("order-created", String.valueOf(event.getProductId()), event)
                .whenComplete((result, ex) -> {
                    if (ex != null) log.error("Failed to send order event", ex);
                    else log.info("Order event sent: partition={}, offset={}",
                        result.getRecordMetadata().partition(),
                        result.getRecordMetadata().offset());
                });
    }
}
```

CONSUMER — OrderStatusConsumer:

```
@Component
public class OrderStatusConsumer {

    @RetryableTopic(attempts = "3",
            backoff = @Backoff(delay = 1000, multiplier = 2),
            dltStrategy = DltStrategy.FAIL_ON_ERROR)
    @KafkaListener(topics = {"payment-success", "payment-failure"},
                   groupId = "order-service-group")
    public void handlePaymentResult(PaymentEvent event) {
        Order order = orderRepository.findById(event.getOrderId()).orElseThrow();
        if ("SUCCESS".equals(event.getStatus())) {
            order.setStatus("CONFIRMED");
        } else {
            order.setStatus("PAYMENT_FAILED");
        }
        orderRepository.save(order);
    }
}
```

5.8 Inventory Service (13 Classes)

PURPOSE: Stock management with two-layer concurrency protection (Redis distributed lock + JPA optimistic locking). The most complex service in terms of concurrency handling.

PROJECT STRUCTURE:

```
inventory-service/
├── pom.xml
└── src/main/java/com/orderprocessing/inventoryservice/
    ├── InventoryServiceApplication.java
    ├── config/
    │   ├── RedisLockConfig.java           # StringRedisTemplate for SETNX
    │   └── SecurityConfig.java
    ├── controller/
    │   ├── InventoryController.java       # /api/inventory/*
```

```
    │   └── AdminInventoryController.java  # /admin/inventory/*
    ├── exception/
    │   └── GlobalExceptionHandler.java
    ├── kafka/
    │   ├── InventoryConsumer.java         # Consumes order/product events
    │   └── InventoryProducer.java         # Produces inventory-updated
    ├── model/
    │   ├── Inventory.java                 # Stock entity with @Version
    │   └── ProcessedEvent.java            # Idempotency table
    ├── repository/
    │   ├── InventoryRepository.java
    │   └── ProcessedEventRepository.java
    └── service/
        └── InventoryService.java          # Lock + reserve + restore logic
```

WHY THIS SERVICE IS COMPLEX:

```
PROBLEM: 100 users buy the last item simultaneously.
  Without protection → overselling (stock goes negative)

SOLUTION: Two-layer concurrency control
  Layer 1: Redis SETNX → only ONE request enters critical section
  Layer 2: JPA @Version → catches edge cases if Redis fails
```

Idempotency Table (ProcessedEvent):

```java
@Entity @Table(name = "processed_events")
public class ProcessedEvent {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    @Column(unique = true, nullable = false)
    private String eventId;        // UUID from Kafka event
    private String eventType;      // "ORDER_CREATED", "ORDER_CANCELLED"
    private LocalDateTime processedAt;
}
```

Consumer — InventoryConsumer (orchestrates the saga step):

```java
@RetryableTopic(attempts = "3", backoff = @Backoff(delay = 1000, multiplier = 2))
@KafkaListener(topics = "order-created", groupId = "inventory-service-group")
public void handleOrderCreated(OrderEvent event) {
    // 1. Idempotency check
    if (processedEventRepository.existsByEventId(event.getEventId())) {
        log.warn("Duplicate event — skipping");
        return;
    }

    // 2. Reserve stock (Redis lock + optimistic locking)
    boolean reserved = inventoryService.reserveStock(event.getProductId(),
event.getQuantity());
```

```
    // 3. Publish result
    if (reserved) {
        inventoryProducer.sendInventoryEvent("inventory-updated",
            new InventoryEvent(event.getOrderId(), "RESERVED"));
    } else {
        inventoryProducer.sendInventoryEvent("payment-failure",
            new InventoryEvent(event.getOrderId(), "INSUFFICIENT_STOCK"));
    }

    // 4. Mark event as processed
    processedEventRepository.save(new ProcessedEvent(event.getEventId(), "ORDER_CREATED"));
}
```

5.9 Payment Service (8 Classes)

PURPOSE: Simulated payment processing. Consumes inventory-updated events, processes payment, and publishes payment-success or payment-failure.

PROJECT STRUCTURE:

```
payment-service/
├── pom.xml
└── src/main/java/com/orderprocessing/paymentservice/
    ├── PaymentServiceApplication.java
    ├── controller/
    │   └── PaymentController.java         # /api/payments/*
    ├── exception/
    │   └── GlobalExceptionHandler.java
    ├── kafka/
    │   ├── PaymentConsumer.java            # Consumes inventory-updated
    │   └── PaymentProducer.java            # Produces payment-success/failure
    ├── model/
    │   └── Payment.java                   # JPA entity → payment_db
    ├── repository/
    │   └── PaymentRepository.java
    └── service/
        └── PaymentService.java
```

PaymentConsumer:

```
@RetryableTopic(attempts = "3", backoff = @Backoff(delay = 1000, multiplier = 2))
@KafkaListener(topics = "inventory-updated", groupId = "payment-service-group")
public void handleInventoryUpdated(InventoryEvent event) {
    if (!"RESERVED".equals(event.getStatus())) return;  // Only process reservations

    boolean paymentSuccess = paymentService.processPayment(event);

    if (paymentSuccess) {
        paymentProducer.sendPaymentEvent("payment-success",
            new PaymentEvent(event.getOrderId(), "SUCCESS", event.getAmount()));
    } else {
```

```
        paymentProducer.sendPaymentEvent("payment-failure",
            new PaymentEvent(event.getOrderId(), "FAILED", event.getAmount()));
    }
}
```

5.10 Notification Service (10 Classes)

PURPOSE: Final step in the saga — consumes payment events for user notifications and audit-log events for system audit trail.

PROJECT STRUCTURE:

```
notification-service/
├── pom.xml
└── src/main/java/com/orderprocessing/notificationservice/
    ├── NotificationServiceApplication.java
    ├── config/
    │   └── SecurityConfig.java
    ├── controller/
    │   ├── NotificationController.java    # /api/notifications/*
    │   └── AdminAuditController.java      # /admin/audit-logs/*
    ├── kafka/
    │   ├── NotificationConsumer.java      # Consumes payment events
    │   └── AuditLogConsumer.java          # Consumes audit-logs
    ├── model/
    │   ├── Notification.java              # JPA entity → notifications
    │   └── AuditLog.java                  # JPA entity → audit_logs
    └── repository/
        ├── NotificationRepository.java
        └── AuditLogRepository.java
```

---

6. Event-Driven Architecture (Kafka)

6.1 Why Event-Driven in This Project?

```
PROBLEM WITH SYNCHRONOUS (REST):
  Order Service → REST → Inventory Service → REST → Payment Service → REST → Notification
  1. Tight coupling: Order Service MUST know Inventory Service URL
  2. Cascade failure: If Payment is down → Order fails → user gets error
  3. Latency: User waits for ALL 4 services to complete (sequential)
  4. Scalability: All services process at the same rate (bottleneck)

SOLUTION WITH EVENT-DRIVEN (KAFKA):
  Order Service → Kafka → (Inventory, Payment, Notification consume independently)
  1. Loose coupling: Order Service doesn't know who consumes its events
  2. Fault tolerance: If Notification is down, Kafka retains messages
  3. Speed: User gets 200 OK immediately (async processing in background)
  4. Scalability: Each consumer scales independently (partitions)
```

6.2 Event Flow Diagram

```
┌──────────────┐   order-created    ┌──────────────────┐
│ Order Service│──────────────────▶│ Inventory Service  │
└──────────────┘                    └──────────┬───────┘
       ▲                                       │
       │  inventory-updated                    │ inventory-updated
       │  (RESERVED/INSUFFICIENT)              │ (RESERVED)
       │                                       ▼
       │                           ┌──────────────────┐
       │                           │ Payment Service    │
       │                           └──────────┬───────┘
       │                                      │
  payment-success                             │ payment-success
  payment-failure                             │ payment-failure
       │                                      │
       └──────────────────────────────────────┘
                   │
                   ▼
          ┌──────────────────┐
          │Notification Service│
          └──────────────────┘
```

6.3 Topic Naming Strategy

```
CONVENTION: {domain-entity}-{action}

+──────────────────+──────────────+──────────────────+──────────────────+
| Topic            | Producer     | Consumer(s)      | Partition Key    |
+──────────────────+──────────────+──────────────────+──────────────────+
| product-created  | Product Svc  | Inventory Svc    | productId        |
| product-events   | Product Svc  | (analytics)      | productId        |
| order-created    | Order Svc    | Inventory Svc    | productId        |
| order-cancelled  | Order Svc    | Inventory Svc    | productId        |
| inventory-updated | Inventory Svc| Order, Payment  | orderId          |
| payment-success  | Payment Svc  | Order, Notif.    | orderId          |
| payment-failure  | Payment Svc  | Order, Notif.    | orderId          |
| audit-logs       | All services | Notification Svc | —                |
+──────────────────+──────────────+──────────────────+──────────────────+

WHY productId AS PARTITION KEY FOR order-created?
  All orders for the SAME product go to the SAME partition.
  Ensures ordering: if 2 orders for product-42 arrive, they are processed sequentially.
  Prevents race condition on stock reservation.
```

6.4 Consumer Groups

```
CONSUMER GROUP STRATEGY:

  Topic: payment-success
    Consumer Group: order-service-group → Order Service reads ALL events
    Consumer Group: notification-service-group → Notification also reads ALL events
```

```
  WHY? Each consumer group gets its OWN copy of every message.
  Order Service updates order status. Notification Service sends email.
  Both can process the same event independently.

  Scaling consumer within group:
    inventory-service-group has 3 instances
    Topic order-created has 6 partitions
    → Each instance handles 2 partitions → parallel processing
```

6.5 Retry & Dead Letter Topic (DLT)

```java
@RetryableTopic(
    attempts = "3",                     // Retry 3 times
    backoff = @Backoff(
        delay = 1000,                   // Initial delay: 1 second
        multiplier = 2),                // Exponential: 1s, 2s, 4s
    dltStrategy = DltStrategy.FAIL_ON_ERROR  // Send to DLT on final failure
)
@KafkaListener(topics = "order-created", groupId = "inventory-service-group")
public void handleOrderCreated(OrderEvent event) { ... }
```

```
RETRY FLOW:
  Attempt 1: Process event → fails → wait 1s
  Attempt 2: Retry → fails → wait 2s
  Attempt 3: Retry → fails → send to DLT

  DLT topic: order-created-dlt (auto-created by Spring Kafka)
  DLT consumer: Manual review / alerting / dead letter processing
```

6.6 Idempotency Handling

```
PROBLEM: Kafka guarantees at-least-once delivery.
  Consumer processes event → crashes BEFORE committing offset
  → Kafka redelivers the same event → consumer processes it AGAIN
  → Duplicate stock reservation! Order counted twice!

SOLUTION: ProcessedEvent table (implemented in Inventory Service)
  Before processing: Check if eventId already exists in processed_events table
  If exists → skip (duplicate)
  If not exists → process → save eventId to processed_events table

  CREATE TABLE processed_events (
      id BIGINT AUTO_INCREMENT PRIMARY KEY,
      event_id VARCHAR(255) UNIQUE NOT NULL,
      event_type VARCHAR(50),
      processed_at TIMESTAMP
  );
```

6.7 At-Least-Once vs Exactly-Once

```
THIS PROJECT USES: At-Least-Once + Application-Level Idempotency

  At-Least-Once (Kafka default):
    ✓ Simple configuration
    ✓ No performance overhead
    ✗ Can deliver duplicates → handled by idempotency table

  Exactly-Once (Kafka Transactions):
    ✓ No duplicates
    ✗ Complex configuration
    ✗ 3-5x slower (transactional overhead)
    ✗ Only works within Kafka ecosystem (not DB)

  OUR APPROACH: At-Least-Once is simpler and sufficient when combined
  with ProcessedEvent idempotency checks.
```

6.8 Kafka Configuration (application.yml)

```yaml
# Producer Configuration (Order Service)
spring:
  kafka:
    bootstrap-servers: localhost:9092
    producer:
      key-serializer: org.apache.kafka.common.serialization.StringSerializer
      value-serializer: org.springframework.kafka.support.serializer.JsonSerializer
      properties:
        spring.json.type.mapping: >              # Type mapping for JSON
          orderEvent:com.orderprocessing.events.OrderEvent
      retries: 3                                 # Retry on transient failures
      acks: all                                  # Wait for all replicas

# Consumer Configuration (Inventory Service)
spring:
  kafka:
    bootstrap-servers: localhost:9092
    consumer:
      group-id: inventory-service-group
      auto-offset-reset: earliest                # Start from beginning if new group
      key-deserializer: org.apache.kafka.common.serialization.StringDeserializer
      value-deserializer: org.springframework.kafka.support.serializer.JsonDeserializer
      properties:
        spring.json.trusted.packages: com.orderprocessing.*  # Trust event packages
        spring.json.type.mapping: >
          orderEvent:com.orderprocessing.events.OrderEvent
```

---

7. Caching Strategy (Redis)

7.1 Where Redis is Used

```
+---------------+-----------------+-----------------+-----------------+
| Service       | Feature         | Redis Command   | Key Pattern     |
+---------------+-----------------+-----------------+-----------------+
| Product Svc   | Application     | GET, SET, DEL   | product::1      |
|               | caching         | TTL: 10/5 min   | products::all   |
+---------------+-----------------+-----------------+-----------------+
| Inventory Svc| Distributed lock | SETNX + EX 30   | inventory:lock: |
|               | (stock reserve) | GET, DEL        | {productId}     |
+---------------+-----------------+-----------------+-----------------+
| API Gateway   | Rate limiting   | INCR, EXPIRE    | (internal)      |
|               | (Resilience4j)  | TTL: 60s        |                 |
+---------------+-----------------+-----------------+-----------------+
```

7.2 Cache Invalidation Strategy

```
CREATE product → @CacheEvict("products")         Clear list
UPDATE product → @CachePut("product") +           Update individual + clear list
                 @CacheEvict("products")
DELETE product → @CacheEvict("product","products") Clear both
TTL expiry     → Auto-expire after 10/5 minutes   Safety net
```

7.3 Distributed Lock — SETNX Pattern

```java
// Atomic lock acquisition
Boolean acquired = redisTemplate.opsForValue()
        .setIfAbsent(lockKey, lockValue, Duration.ofSeconds(30));
//  SET inventory:lock:42 "uuid-abc" NX EX 30

// Safe release (only if WE hold the lock)
String currentValue = redisTemplate.opsForValue().get(lockKey);
if (lockValue.equals(currentValue)) {
    redisTemplate.delete(lockKey);
}
```

---

8. Security Architecture (JWT)

8.1 JWT Token Structure

```
HEADER.PAYLOAD.SIGNATURE

Header:  {"alg": "HS256", "typ": "JWT"}
Payload: {"sub": "john", "role": "USER", "iat": 1714695423, "exp": 1714781823}
Signature: HMACSHA256(base64(header) + "." + base64(payload), secretKey)

Token lifetime: 24 hours (configurable in JwtUtil)
Secret key: Shared between Auth Service and API Gateway
```

8.2 JWT Flow in This System

```
1. User → POST /api/auth/login {username, password}
   → Auth Service validates credentials → generates JWT
   → Returns: {"token": "eyJhbG...", "role": "USER"}

2. User → GET /api/products (Authorization: Bearer eyJhbG...)
   → API Gateway: JwtAuthenticationFilter
      → Extract token from "Bearer " prefix
      → Validate signature using shared secret
      → Check expiry: exp > current time
      → Extract claims: username="john", role="USER"
      → Add headers: X-User-Name: john, X-User-Role: USER
      → Forward to Product Service

3. Product Service reads X-User-Name header
   → Knows the request is from "john"
   → No JWT parsing needed in downstream services
```

8.3 Role-Based Access Control

```
ENDPOINT SECURITY MATRIX:

+———————————————————+————————+————————+————————+
| Endpoint          | No Auth | USER   | ADMIN  |
+———————————————————+————————+————————+————————+
| POST /api/auth/signup|   ✓    |   ✓    |   ✓    |
| POST /api/auth/login |   ✓    |   ✓    |   ✓    |
| GET /api/products    |   ✗    |   ✓    |   ✓    |
| POST /admin/products |   ✗    |   ✗    |   ✓    |
| POST /api/orders     |   ✗    |   ✓    |   ✓    |
| GET /admin/orders    |   ✗    |   ✗    |   ✓    |
| GET /admin/audit-logs|   ✗    |   ✗    |   ✓    |
+———————————————————+————————+————————+————————+
```

8.4 SecurityConfig Pattern

```java
// Used in ALL services (except Auth Service and Gateway)
@Configuration @EnableWebSecurity @EnableMethodSecurity
public class SecurityConfig {
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) {
        return http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/**").permitAll()     // Public endpoints
                .requestMatchers("/admin/**").hasRole("ADMIN") // Admin only
                .anyRequest().authenticated())
            .build();
```

```
    }
}
```

WHY permitAll for /api/?** JWT validation happens at the Gateway level (JwtAuthenticationFilter). By the time the request reaches the service, the user is already authenticated. The X-User-Role header can be used for `@PreAuthorize` checks. This avoids duplicate JWT validation.

---

9. Design Patterns Used

9.1 Repository Pattern

```
WHAT: Abstracts data persistence behind an interface.
WHERE: Every service — ProductRepository, OrderRepository, etc.
WHY: Services don't care about SQL queries — they call findById(), save(), etc.

Spring Data JPA generates the implementation:
  public interface ProductRepository extends JpaRepository<Product, Long> {
      List<Product> findByActiveTrue();
      Optional<Product> findByName(String name);
  }
```

9.2 DTO (Data Transfer Object) Pattern

```
WHAT: Separate object for API communication (never expose entities directly).
WHERE: ProductDto, OrderRequest, OrderResponse, UserProfileDto, AuthRequest, etc.
WHY:
  1. Security — don't expose @Version, password hash, or internal IDs accidentally
  2. Flexibility — API shape can differ from DB schema
  3. Validation — DTOs have @NotNull, @Size, @Email annotations

Entity (internal):   Product { id, name, price, active, createdAt, updatedAt }
DTO (external):      ProductDto { id, name, price }
```

9.3 Builder Pattern

```
WHAT: Construct complex objects step by step.
WHERE: Event classes via @Builder (Lombok), RedisCacheConfiguration.builder()
WHY: Events have 8+ fields — constructor with 8 params is unreadable.

  OrderEvent event = OrderEvent.builder()
      .eventId(UUID.randomUUID().toString())
      .orderId(order.getId())
      .productId(order.getProductId())
      .quantity(order.getQuantity())
      .totalAmount(order.getTotalAmount())
      .status("PENDING")
      .username(username)
      .timestamp(LocalDateTime.now())
      .build();
```

9.4 Observer Pattern (via Kafka)

```
WHAT: One-to-many dependency — when subject changes, all observers are notified.
WHERE: Kafka producers/consumers throughout the system.
WHY: Order Service doesn't know about Inventory, Payment, or Notification.

  Subject/Publisher: Order Service publishes order-created
  Observers/Subscribers:
    - Inventory Service (reserves stock)
    - (any future service can subscribe without modifying Order Service)

  BENEFIT: New services (e.g., Analytics) can subscribe to order-created
  without any changes to Order Service — true Open/Closed Principle.
```

9.5 Saga Pattern (Choreography)

```
WHAT: Distributed transaction across multiple services using events.
WHERE: Order → Inventory → Payment → Notification flow.
WHY: Traditional ACID transactions don't work across microservices (different DBs).

  THIS PROJECT USES CHOREOGRAPHY SAGA:
    Each service listens to events and publishes its own events.
    No central coordinator — services react to events independently.

  SAGA STEPS:
    1. Order Service → PENDING → publishes order-created
    2. Inventory Service → reserve stock → publishes inventory-updated (RESERVED)
    3. Payment Service → process payment → publishes payment-success
    4. Order Service → CONFIRMED (saga complete ✓)

  COMPENSATION (rollback):
    2. Inventory Service → insufficient stock → publishes payment-failure
    → Order Service → CANCELLED
    → Notification Service → notifies user "order failed"

  ALTERNATIVE: Orchestration Saga (central coordinator like Order Saga Service)
    Pros: Explicit flow control, easier debugging
    Cons: Single point of failure, more complex service
```

9.6 Circuit Breaker Pattern

```
WHAT: Prevent cascading failures by stopping calls to a failing service.
WHERE: API Gateway — Resilience4j Circuit Breaker per downstream service.
WHY: If Product Service is down, don't keep sending requests (they'll all fail).

  States:
    CLOSED (normal): Requests pass through. Track failure rate.
    OPEN (circuit tripped): ALL requests fail immediately (return fallback).
    HALF-OPEN (testing): Allow limited requests to test recovery.
```

```
  Configuration:
    failure-rate-threshold: 50%
    wait-duration-in-open-state: 10s
    sliding-window-size: 10 requests

  Fallback:
    @RestController public class FallbackController {
        @GetMapping("/fallback/generic")
        public ResponseEntity<Map> fallback() {
            return ResponseEntity.status(503).body(Map.of(
                "message", "Service temporarily unavailable. Please try again."));
        }
    }
```

9.7 Factory Pattern (via Spring)

```
WHAT: Spring IoC container acts as a factory — creates and manages beans.
WHERE: @Configuration classes, @Bean methods, @Component scanning.
WHY: CacheManager, SecurityFilterChain, KafkaTemplate — all created by factories.

  @Bean
  public CacheManager cacheManager(RedisConnectionFactory factory) {
      // Factory method — creates the RedisCacheManager with specific config
      return RedisCacheManager.builder(factory)
              .cacheDefaults(defaultConfig)
              .withCacheConfiguration("products", listConfig)
              .build();
  }
```

9.8 Singleton Pattern

```
WHAT: One instance per class in the application context.
WHERE: Every @Service, @Component, @Repository, @Controller (Spring default scope).
WHY: ProductService bean is created once and reused for all requests.
  Ensures thread-safety with stateless design (no mutable instance fields).
```

9.9 Strategy Pattern (via SecurityConfig)

```
WHAT: Define a family of algorithms, encapsulate each, make them interchangeable.
WHERE: SecurityFilterChain — different security strategies per service.

  Auth Service strategy: permitAll for /api/auth/** (public login/register)
  Product Service strategy: permitAll for /api/**, ADMIN for /admin/**
  API Gateway strategy: JWT filter for protected routes, skip for auth routes

  Each service defines its OWN SecurityFilterChain bean with different rules.
```

9.10 Outbox Pattern (Recommended Enhancement)

```
WHAT: Prevent data inconsistency between DB write and Kafka publish.
WHERE: Could be added to Order Service and Inventory Service.

PROBLEM:
  1. Save order to DB ✓
  2. Publish event to Kafka ✗ (Kafka temporarily down)
  Result: Order exists in DB but no event sent → inventory never reserved

SOLUTION (Outbox Pattern):
  1. Save order to DB ✓
  2. Save event to "outbox" table in SAME DB ✓ (single transaction)
  3. Background process reads outbox table → publishes to Kafka → marks as sent

  This project doesn't implement Outbox yet but it's the recommended production fix.
```

---

10. Docker & Deployment

10.1 Dockerfile Pattern (Used by All Services)

```
# Multi-stage build for minimal image size
FROM eclipse-temurin:17-jdk-alpine AS builder
WORKDIR /app
COPY target/*.jar app.jar

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=builder /app/app.jar app.jar

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

WHY multi-stage?

- Builder stage: has full JDK (400MB) for compilation
- Runtime stage: only JRE (~150MB) for execution
- Final image: ~180MB vs 400MB+ (55% smaller)

10.2 Docker Compose Architecture

```
# 4 infrastructure containers + 8 service containers + 1 frontend = 13 total
services:
  # Level 0: Infrastructure
  mysql:          (port 3307→3306, healthcheck, volumes)
  zookeeper:      (port 2181)
  kafka:          (port 9092, depends on zookeeper)
  redis:          (port 6379, volumes)

  # Level 1: Core
  eureka-server:  (port 8761, healthcheck)
```

```
  # Level 2: Gateway
  api-gateway:    (port 8088→8080, depends on eureka + redis)

  # Level 3: Services
  auth-service:   (depends on mysql + eureka)
  user-service:   (depends on mysql + eureka)
  product-service:(depends on mysql + eureka + redis)
  order-service:  (depends on mysql + eureka + kafka)
  inventory-svc:  (depends on mysql + eureka + kafka + redis)
  payment-service:(depends on mysql + eureka + kafka)
  notification:   (depends on mysql + eureka + kafka)

  # Level 4: Frontend
  frontend:       (port 3000→80, depends on gateway)
```

10.3 Environment Variable Strategy

```
RULE: Never hardcode infrastructure addresses.
  WRONG: spring.datasource.url=jdbc:mysql://localhost:3306/order_db
  RIGHT: SPRING_DATASOURCE_URL=jdbc:mysql://mysql:3306/order_db

Docker injects environment variables that override application.yml.
  application.yml: spring.data.redis.host=localhost    (for local dev)
  Docker env:      SPRING_DATA_REDIS_HOST=redis        (overrides in Docker)
```

10.4 Startup & Shutdown

```
# Full system startup
docker compose up -d

# Shutdown (preserve data)
docker compose down

# Shutdown (delete all data)
docker compose down -v

# View logs
docker compose logs -f order-service

# Scale a service
docker compose up -d --scale order-service=3
```

---

11. Production Considerations

11.1 Logging Strategy

```
PATTERN: SLF4J + Logback (Spring Boot default)
CONVENTION: Log at service boundaries (controller entry, Kafka events, errors)
```

```
  log.info("Order created: orderId={}, productId={}", order.getId(), productId);
  log.warn("Insufficient stock for productId={}", productId);
  log.error("Payment processing failed for orderId={}", orderId, exception);

PRODUCTION ENHANCEMENT:
  - Add Correlation ID (tracing across services)
  - Structured logging (JSON format for ELK stack)
  - Centralized log aggregation (ELK or Grafana Loki)
```

11.2 Health Checks

```yaml
# Spring Boot Actuator (enabled in all services)
management:
  endpoints:
    web:
      exposure:
        include: health, info, metrics
  health:
    kafka:
      enabled: true           # Check Kafka connectivity
    redis:
      enabled: true           # Check Redis connectivity
    db:
      enabled: true           # Check MySQL connectivity
```

11.3 Rate Limiting

```
WHERE: API Gateway (Resilience4j)
CONFIGURATION:
  limit-for-period: 100        # 100 requests per window
  limit-refresh-period: 1m     # Window = 1 minute
  timeout-duration: 0s         # Reject immediately (no queuing)
BACKED BY: Redis (shared state across gateway instances)
```

11.4 Graceful Shutdown

```yaml
# Spring Boot graceful shutdown
server:
  shutdown: graceful
spring:
  lifecycle:
    timeout-per-shutdown-phase: 30s   # Wait 30s for in-flight requests
```

11.5 Scaling Strategy

```
HORIZONTAL SCALING:
  docker compose up --scale order-service=3
  → Eureka registers 3 instances
```

```
  → Gateway load-balances (round-robin) across all 3

KAFKA SCALING:
  Increase partitions: kafka-topics --alter --topic order-created --partitions 6
  Add consumer instances: --scale inventory-service=6
  → 6 consumers × 6 partitions = maximum parallelism

REDIS SCALING:
  Development: single instance (current)
  Production: Redis Cluster (3 masters + 3 replicas)
```

---

12. Common Pitfalls & Solutions

#	Pitfall	Symptom	Solution
1	Forgetting to build common-events first	`ClassNotFoundException: OrderEvent`	`cd common-events && mvn install` first
2	MySQL not healthy when services start	`Connection refused` on startup	Use `depends_on: condition: service_healthy`
3	Kafka listeners not matching producer types	`ClassCastException` in consumer	Ensure `spring.json.type.mapping` matches
4	JWT secret mismatch between Auth & Gateway	All requests return 401	Same secret key in Auth Service and Gateway
5	CORS blocking frontend requests	Browser shows `CORS error`	CorsConfig in Gateway allows `localhost:3000`
6	Cache returning stale data after update	User sees old data	Use @CacheEvict + @CachePut on mutations
7	Duplicate Kafka event processing	Stock reserved twice	Implement idempotency with ProcessedEvent table
8	Redis lock not released on exception	Subsequent requests blocked	Always release lock in `finally` block
9	Order status stuck at PENDING	Order never updates	Check Kafka consumer group offset lag
10	Running out of Kafka consumer threads	Slow message processing	Increase `concurrency` property on listener
11	Hot key in Redis cache	Single product overloads Redis	Add L1 Caffeine cache in front of Redis
12	No circuit breaker fallback defined	500 errors propagate to user	Add FallbackController with friendly message

---

13. Interview Questions

Architecture & Design

Q1: Why did you choose microservices over monolithic?

We needed: (1) Independent deployment — Product Service can be deployed without affecting Payment Service. (2) Independent scaling — Order Service handles 10x more traffic than User Service. (3) Technology flexibility — Gateway uses reactive WebFlux, other services use MVC. (4) Team autonomy — 7 teams can work on 7 services independently. (5) Fault isolation — Payment Service crash doesn't bring down Product catalog.

Q2: How does your order processing handle distributed transactions?

We use the **Choreography Saga pattern**. There's no distributed ACID transaction. Instead: (1) Order Service saves order (PENDING) → publishes order-created. (2) Inventory Service reserves stock → publishes inventory-updated. (3) Payment Service processes payment → publishes payment-success. (4) Order Service updates status to CONFIRMED. If any step fails, compensation events are published (payment-failure → Order CANCELLED → stock restored). Each service manages its own database transaction.

Q3: Why is your JWT validated at the Gateway, not in every service?

(1) Single point of validation — one JWT implementation to maintain. (2) Downstream services are simpler — they just read X-User-Name header. (3) Performance — JWT signature verification happens once, not 7 times. (4) Secret management — only Gateway and Auth Service need the JWT secret. (5) Services behind the Gateway trust internal headers (network isolation via Docker).

Q4: How do you prevent overselling in your inventory system?

Two-layer protection: (1) **Redis SETNX lock** — only one request per product enters the critical section. Lock has 30s TTL (auto-release on crash). Lock value is UUID (only the holder can release). (2) **JPA @Version field** — optimistic locking catches edge cases where Redis lock fails. If version changed between read and write → retry up to 3 times with exponential backoff.

Q5: What happens when a service goes down?

(1) **API Gateway Circuit Breaker** — detects failures, opens circuit, returns fallback response (503 with friendly message). (2) **Kafka message retention** — if Inventory Service is down, order-created events wait in Kafka until it recovers. (3) **Eureka deregistration** — Gateway stops routing to dead instances. (4) **Redis degradation** — if Redis dies, cache falls back to DB (slower but functional). Locks fail safely (return false → reject operation).

Q6: Explain your database-per-service pattern.

Each service owns its database: auth_db, user_db, product_db, order_db, inventory_db, payment_db, notification_db. Benefits: (1) Schema independence — Order Service can change its schema without affecting Product Service. (2) Technology flexibility — one service could switch to PostgreSQL without affecting others. (3) Scaling — hot databases can be moved to dedicated servers. (4) Fault isolation — if inventory_db is locked, product_db is unaffected.

Kafka Questions

Q7: How do you handle duplicate Kafka messages?

We use a **ProcessedEvent table** for idempotency. Every Kafka event has a UUID `eventId`. Before processing, we check: `processedEventRepository.existsByEventId(event.getEventId())`. If true → skip. If false → process → save eventId. This handles Kafka's at-least-once delivery guarantee where messages can be replayed after consumer crashes.

Q8: Why did you choose Kafka over RabbitMQ?

(1) **Event log** — Kafka retains events for days (replay capability). RabbitMQ deletes after delivery. (2) **Throughput** — Kafka handles millions of events/sec via partitions. (3) **Consumer groups** — multiple independent consumers of the same event (Order + Notification both read payment-success). (4) **Ordering** — partition key (productId) guarantees ordering per product.

Redis Questions

Q9: Why separate Redis caching from database caching?

(1) **Performance** — Redis: 0.5ms latency vs MySQL query cache: 2-5ms. (2) **Shared cache** — multiple Product Service instances share the same Redis cache. (3) **Granular control** — custom TTL per cache, JSON serialization, manual eviction. (4) **Additional features** — distributed locks, rate limiting (not possible with MySQL cache).

---

14. Future Enhancements

Enhancement	Benefit	Complexity
Outbox Pattern	Guarantee DB+Kafka consistency	Medium
Distributed Tracing (Zipkin/Jaeger)	Trace requests across services	Low
Centralized Logging (ELK Stack)	Aggregate logs from all services	Medium
Multi-tier Cache (Caffeine + Redis)	99% cache hit rate (L1 in-JVM)	Low
Redis Cluster	High availability + horizontal scaling	Medium
Kafka Schema Registry	Schema evolution + compatibility checks	Medium
Config Server (Spring Cloud Config)	Centralized configuration management	Low
gRPC Inter-service	Replace REST for internal sync calls (faster)	High
Kubernetes Deployment	Production orchestration with auto-scaling	High
API Versioning	`/api/v2/products` without breaking v1 clients	Low
CQRS	Separate read/write models for Order queries	High
Event Sourcing	Full audit trail with event replay	High
Refresh Token	Long-lived sessions without re-login	Medium
OAuth2/OpenID Connect	External identity provider integration	High

---

End of Document

This guide covers the complete architecture, implementation, and operational aspects of the Real-Time Order Processing System. Follow the Development Order Strategy (Section 3) to build the system from scratch, implement each service using the class-by-class guides (Section 5), and refer to the design patterns (Section 9) and production considerations (Section 11) for enterprise-grade quality.